

Chapitre 3 : Gestion des INTERRUPTIONS

Objectifs : Ce que vous allez apprendre à la fin de ce chapitre :

Objectifs Généraux : Les interruptions sont un élément clé des systèmes d'exploitation. Etudier les interruptions dans les systèmes d'exploitation pourrait avoir plusieurs objectifs, notamment :

- Comprendre le rôle fondamental des interruptions dans un système d'exploitation,
- Identifier les différents types d'interruptions et leurs causes,
- Expliquer le mécanisme de gestion des interruptions et le rôle du contrôleur d'interruption,
- Analyser l'impact des interruptions sur les performances et la stabilité du système,
- Comprendre le concept du **mot d'état du processeur** et son rôle dans la gestion des interruptions,
- Expliquer l'utilisation de l'**instruction LPSW** dans la restauration du contexte après une interruption.

Objectifs Spécifiques

1. Définition et Principes de Base
 - Définir une interruption et expliquer son rôle dans un système informatique,
 - Différencier les interruptions **matérielles** et **logicielles**.
2. Mécanisme de Gestion des Interruptions
 - Expliquer comment une interruption est générée et traitée par le processeur,
 - Décrire le fonctionnement du **vecteur d'interruptions** et du **contrôleur d'interruptions**,
 - Comprendre la notion de **priorité des interruptions** et la gestion des interruptions simultanées,
 - Analyser les effets des **interruptions en cascade** et les techniques pour les gérer.
3. Le Mot d'État du Processeur (Processor Status Word - PSW)
 - Définir le **mot d'état du processeur (PSW)** et expliquer son rôle dans la sauvegarde du **contexte**,
 - Identifier les **bits de contrôle** du PSW et leur influence sur le fonctionnement du système,
 - Expliquer comment le processeur **désactive et réactive** les interruptions à travers le PSW.
4. L'Instruction LPSW et la Restauration du Contexte
 - Décrire le rôle de l'instruction **LPSW (Load Process Status Word)**,
 - Expliquer comment LPSW permet de restaurer l'état du processeur après le traitement d'une interruption,
 - Analyser un exemple d'utilisation de LPSW dans la gestion des interruptions sur un processeur spécifique.
5. Types d'Interruptions
 - Identifier et analyser les interruptions **synchrones** (traps, exceptions) et **asynchrones** (interruptions matérielles),
 - Expliquer le rôle des **appels système (SVC - Supervisor Call)** et leur importance dans la gestion des ressources du système.
6. Gestion des Interruptions dans un Système d'Exploitation
 - Expliquer les stratégies de **masquage et d'armement des** interruptions,
 - Décrire le rôle du **registre d'interruption** et ses interactions avec le PSW,

- Comprendre l'importance de la **table des vecteurs d'interruptions** pour le traitement efficace des interruptions.
7. Applications Pratiques et Optimisation
- Expliquer comment les interruptions sont utilisées dans la **gestion des périphériques et des entrées/sorties (I/O)**,
 - Identifier les problèmes courants liés aux interruptions et proposer des solutions pour améliorer la fiabilité du système.

1. Introduction :

Dans un système informatique, les interruptions jouent un rôle fondamental en permettant au processeur de gérer efficacement plusieurs tâches en parallèle. Elles offrent un mécanisme permettant aux périphériques, aux logiciels et aux événements internes du processeur d'interrompre l'exécution normale d'un programme pour traiter une tâche urgente avant de reprendre le travail initial.

Les interruptions permettent ainsi d'optimiser l'utilisation du processeur en évitant les cycles d'attente inutiles. Elles sont essentielles pour la gestion des **entrées/sorties (I/O)**, la **synchronisation des processus**, la **gestion des erreurs** et les **appels système**.

Ce chapitre explore les **différents types d'interruptions**, leur mode de fonctionnement et leur **hiérarchisation** dans le système. Nous étudierons également le **mot d'état du processeur (PSW)**, qui permet de sauvegarder l'état du processeur lors d'une interruption, ainsi que l'**instruction LPSW**, qui facilite la restauration du contexte d'exécution. Enfin, nous analyserons l'impact des interruptions sur les performances du système et les stratégies mises en place par les systèmes d'exploitation pour les gérer efficacement.

2. Concepts de base :

Etudier les interruptions dans les systèmes d'exploitation est important pour comprendre le fonctionnement interne du système, améliorer les performances, développer des pilotes de périphériques efficaces, et résoudre les problèmes de stabilité.

Dans un ordinateur, ils co-existent deux types de programmes :

- Les programmes **usagers** qui font un calcul utile.
- Les programmes du **SE** qui font un travail de superviseur de tous les événements qui arrivent à la machine.

Ces deux types de programmes se partagent les ressources communes de la machine, notamment le processeur. En exécutant un programme, deux modes peuvent avoir lieu :

- **Mode Utilisateur (*Esclave*)** : ce mode se caractérise par un accès restreint aux ressources du système, certaines instructions sont interdites (pas d'accès direct aux périphériques). Il peut être interrompu par d'autres processus. Mais à travers des appels systèmes (**SVC**), un processus en mode utilisateur peut accéder (temporairement) à des fonctions nécessitant le mode Maître.
- **Mode Superviseur ou *Maître* (ou *Noyau*)** : Il a un accès sans restriction (manipulation de la mémoire, dialogue avec les contrôleurs de périphériques, etc.). Le Noyau ou Superviseur est l'entité système permettant la gestion des processus. Seul le noyau a un accès direct à toutes les ressources, il regroupe la gestion des E/S, l'ordonnancement des processus, la gestion de la mémoire, la gestion des interruptions.

Un processus utilisateur souhaitant accéder à un périphérique devra passer tout d'abord par un appel-système (SVC). Un appel-système est une requête traitée par le noyau pour le compte d'un processus. La notion de noyau et d'appel-système permettent de protéger les données sensibles du système, car seul ce dernier peut les modifier. De plus, un appel-système n'est pas interruptible, ce qui fait qu'il n'y a pas de concurrence entre ceux-ci.

Lorsqu'un programme est en cours d'exécution, plusieurs événements peuvent survenir, à savoir :

- Les événements **Synchrones** qui sont liés à l'exécution du programme en cours d'exécution, ce sont de deux types :
 1. Les **déroutements** qui sont provoqués par des déroulements anormaux des instructions du programme : Division par zéro, débordement d'une zone mémoire, Exécution d'une instruction inexistante ou interdite, Tentative d'accès à une zone protégée, ...etc.
 2. Les **SVC** correspondent à des appels à des fonctions du système d'exploitation, comme par exemple des demandes d'E/S.
- Les événements **Asynchrones**, ce sont des événements extérieurs qui doivent être pris en compte et qui viennent interférer de façon imprévisible avec le programme en cours de déroulement normal. Il s'agit en particulier :
 1. Fin d'opération d'E/S.
 2. Signal d'horloge (par exemple dans le cas du quantum de temps).

*Dans les anciens systèmes, la supervision ou le contrôle de ces deux types d'événements **se faisait** par un contrôle continu du SE et cela à chaque instant d'arrivée de ces évènements.*

- **Problème** : Par quel mécanisme peut-on réduire ce temps de *supervision* ?
- **Solution** : Au lieu que ça soit le processeur qui contrôle continuellement l'état d'une ressource, c'est plutôt la ressource qu'elle doit informer le processeur sur son état au moment significatif. Cette information est transmise via un signal géré par ce que l'on appelle le mécanisme de gestion **d'interruptions** des programmes.

3. Définition et Mécanisme de base :

3.1. Définition :

Une interruption est un signal déclenché par un événement externe ou interne à la machine, et qui provoque **l'arrêt immédiat** d'un programme en cours d'exécution, et cela juste à la fin de l'opération courante, plus particulièrement dans un point que l'on appelle **point observable** ou interruptible du processeur (c'est un point entre l'exécution de deux opérations élémentaires ¹), et cela au profit d'un programme plus prioritaire appelé programme d'interruption (ou **Traitant de l'interruption**). Ensuite, le programme interrompu reprend son exécution à l'endroit où il avait été interrompu.

Physiquement, l'interruption se traduit par un signal envoyé au processeur. Elle permet de **forcer** le processeur à suspendre l'exécution du programme en cours, et à déclencher l'exécution d'un autre programme prédéfini, spécifique à l'événement (appelé la cause de l'interruption) et qui permet d'exécuter le traitement de l'interruption ; ce dernier peut avoir plusieurs appellations : **Traitant de l'interruption (Interrupt-Handler)** ou **Rit (Routine d'interruption)**, ou encore **ISR (Interrupt Service Routine)**.

Le traitant de l'interruption fait d'abord une sauvegarde du contexte du processus interrompu avant de réaliser son propre traitement. A la fin de celui-ci, le contexte du processus interrompu est restauré ce qui lui permet de continuer son exécution convenablement à l'endroit où il a été interrompu.

Les interruptions sont les **sonnettes** que tirent les périphériques pour dire au processeur que quelque chose arrive. Mais si chaque périphérique pourrait envoyer directement un signal au processeur, il faudrait avoir sur celui-ci autant de broches (jouant le rôle de cordons de sonnette) que de périphériques. Pour éviter cela, les périphériques envoient leur requête d'interruption à une puce appelée **contrôleur d'interruption** à laquelle ils sont connectés. C'est ce contrôleur-là qui va envoyer à leur place une interruption au processeur.

¹ Opération machine.

Les requêtes faites par le contrôleur sont appelées **IRQ** (Interrupt Request). Le contrôleur d'interruption est le dispositif incorporé au séquenceur qui détecte les signaux d'interruption. Ces signaux arrivent de façon asynchrone, à n'importe quel moment, mais ils ne sont pris en compte qu'à la **fin** de l'opération en cours.

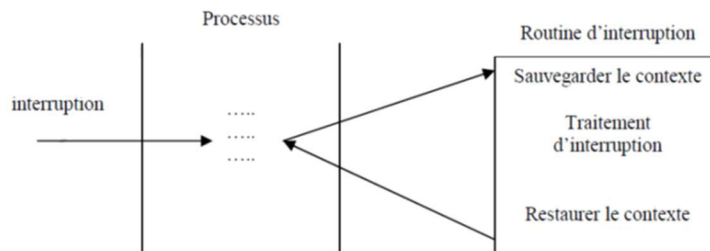


Figure 1 : Déroulement d'une routine d'interruption

3.2. Schéma général d'un programme d'interruption : Une interruption (en général) a pour fonction de forcer un processeur à réagir face à un événement.

==> L'exécution du programme en cours est suspendue et un programme d'interruption est exécuté. Ensuite le programme repris par le processeur peut être le programme interrompu ou un autre programme (voir figure 2).

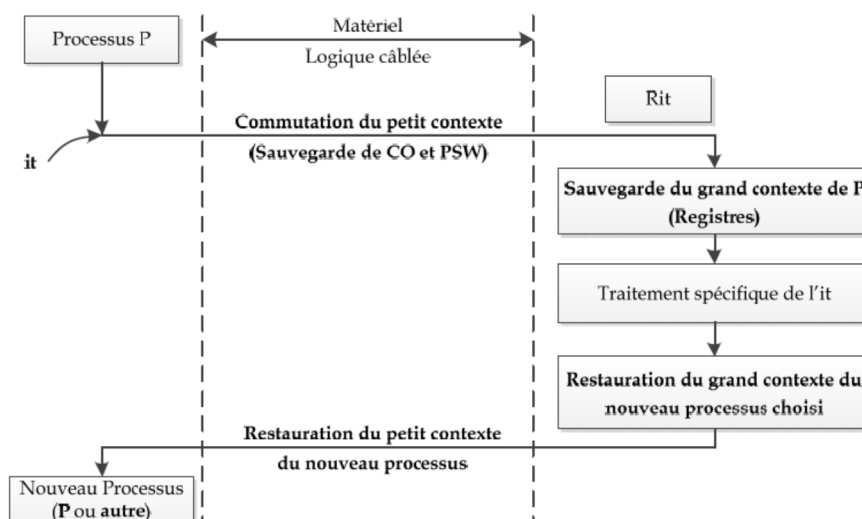


Figure 2 : Mécanisme de gestion d'une interruption

3.3. Structure générale d'un Traitant (Rit) : Un traitant d'une interruption, c'est un programme qui gère la cause de l'interruption, il a toujours la structure suivante :

Traitant _It_i:

1. Sauvegarder la valeur des registres généraux (dans un emplacement particulier de la mémoire, appelé par défaut **zone_i**). Cette étape est couramment appelée la sauvegarde du grand contexte, qui est réalisée par l'instruction_système **Sauver (zone_i)**.
2. Traiter la cause de l'interruption (c'est le programme spécifique de l'interruption).
3. Restaurer la valeur des registres généraux (grand contexte) par l'instruction **Restaurer (zone_i)**.
4. Restaurer le mode du programme interrompu et charger dans le compteur ordinal la valeur préalablement sauvegardée, là où le programme a été interrompu, et cela par l'instruction : **Lpsw (ancien-mep-It_i)** qui signifie load_MEP qui est utilisée pour charger un nouveau PSW, permettant ainsi de gérer correctement la reprise du processus interrompu et elle provoque le retour automatique au programme interrompu. *Cette étape est appelée l'acquittement de l'interruption.*

4. Notions de base :

4.1.Contexte et mot d'état du processeur : Le contexte d'un processeur est constitué de l'ensemble des informations qui définissent son état à un instant donné, étant :

Grand Contexte	Petit Contexte
registres généraux	registres spéciaux : MEP

- les registres généraux : Ils sont adressables et modifiables par l'utilisateur (c'est à dire par des programmes);
- les registres spéciaux : ils sont inaccessibles directement par l'utilisateur et sont regroupés dans le **MEP**, Mot d'Etat du Processeur ; en anglais **PSW**, Processor Status Word.

Le MEP est un élément clé pour stocker l'état de la CPU avant une interruption et le restaurer après, il contient les quatre champs suivants:

Activité	Mode	Masquage	Compteur Ordinal : CØ
-----------------	-------------	-----------------	------------------------------

- **l'activité ou l'état d'exécution du processeur** (actif ou en attente). Le processeur dans l'état actif peut exécuter et enchaîner les instructions. Le changement d'état **Actif ==>Attente** est réalisé par le processeur lui-même, soit par l'exécution d'une instruction spécialisée, soit par (auto)-modification du MEP. Le passage **Attente ==>Actif** ne peut évidemment pas être effectué par le processeur

lui-même puisqu'il est déjà en attente. Il ne peut être obtenu que par un événement extérieur (une **interruption**).

- le **mode** (maître ou esclave). Comme nous l'avons déjà vu, il y a deux modes d'exécution, le **mode utilisateur** (ou mode **Esclave** ou mode **User**) et le mode noyau ou **mode superviseur** (mode **maître** ou mode **kernel**). Les instructions privilégiées comprennent en particulier les instructions d'E/S, celles touchant aux interruptions (armement, désarmement, masquage, démasquage, déclenchement, ainsi que celles touchant à la sécurité et à la protection).
- Le **masque** d'interruption (Masqué ou Démasqué). Lorsque les interruptions sont masquées, le processeur ne peut être interrompu, jusqu'à ce qu'il les démasque. Comme il y a plusieurs types d'interruptions, chacune d'entre elles peut être masquée/démasquée. L'ensemble des états de chaque type d'interruptions constitue un **masque d'interruptions**.
- le **compteur ordinal** : C'est un registre non programmable qui contient l'adresse de la prochaine instruction à exécuter.

4.2. Commutation de contexte :

Lorsqu'une interruption it_i survient ; une commutation de contexte **automatique** s'effectue par **hardware** et qui permet en une seule opération indivisible de (voir figure 3):

(a) Ranger dans un emplacement spécifié de la mémoire, le contenu courant du mot d'état du processus, c'est dans l' **Ancien_mep_it_i**

(b) Charger dans le mot d'état du processeur un nouveau contenu préparé à l'avance dans un emplacement spécifié de la mémoire (c'est le **Nouveau_mep_it_i**), qui correspond à l'interruption. Le compteur ordinal donc est écrasé, et la nouvelle valeur qui est l'adresse du code du Traitant de l'interruption. Ce dernier sera alors exécuté et la première chose qu'il va faire est de finaliser l'opération de sauvegarde du grand contexte (registres généraux) du processus dans une zone mémoire, par une instruction que nous appellerons symboliquement **Sauver(Zone ou PCB)**.

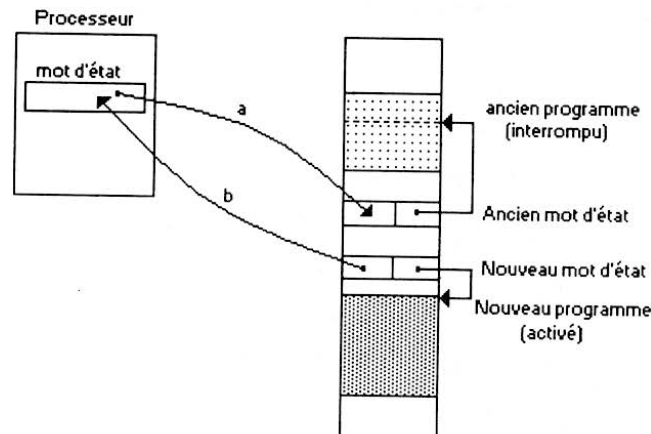


Figure 3 : Mécanisme de commutation de contexte

- **NB** : Le temps pour faire une commutation de contexte est appelé **l'overhead²** dans lequel le SE ne fait pas de travail utile (temps dépendant du support matériel généralement négligeable).

4.3. Vecteur d'interruptions :

Lors de l'occurrence d'une interruption, le processeur reçoit un index qui pointe dans le vecteur d'interruptions. Cet index est généré par le processeur dans le cas des déroutements (interruption logicielle) et il est fourni dans le cas d'une interruption matérielle par un circuit de la carte mère et qui est appelé **contrôleur d'interruption** (pour plus de détails, voir plus loin). Grâce à cet index, le processeur accède à l'entrée correspondante dans la table d'interruption (voir figure 4).

Lorsque le signal d'une interruption arrive, il modifie l'état d'un indicateur (*Drapeau* ou **Flag**) dans le **registre d'interruptions** qui est régulièrement testé par l'unité centrale. Une fois que le signal est détecté, il faut déterminer la cause de l'interruption. Pour cela, on utilise un indicateur, pour les différentes causes d'interruption. On associe à chaque cause d'interruption un numéro **k** qui l'identifie. On dispose également dans la zone basse de la mémoire centrale d'une table appelée **Vecteur d'interruptions (vi)**. Les cases $vi[k]$ de cette table contiennent l'adresse de la routine à exécuter lors de l'arrivée d'une interruption de cause **k**. Cette routine est appelée le **traitant d'interruption** de cause **k**.

² **Overhead** : L'overhead est le temps (nécessaire) pris par le système d'exploitation pour exécuter ses fonctions internes. On peut citer comme exemples : les fonctions d'ordonnancement de programmes et de processus, le traitement des interruptions, le traitement des trappes ... Un bon système tend à minimiser son overhead pour pouvoir allouer un maximum de temps UC aux programmes utilisateur.

Un vecteur d'interruption a donc la structure suivante :

N° d'interruption	@ du traitant
0	@_traitant_it ₀
1	@_traitant_it ₁
2	@_traitant_it ₂
3	@_traitant_it ₃
N	@_traitant_it _N

Figure 4 : Vecteur d'interruptions

Le schéma de la Figure 5 donne un aperçu de la structure et du contenu de la Zone système, après chargement des routines d'interruption et initialisation des vecteurs d'interruptions. **La Table d'Interruptions**, située à l'adresse 0 de la mémoire, regroupe toutes les adresses des traitants d'interruptions.

vi[k] représente le traitant de l'interruption i.

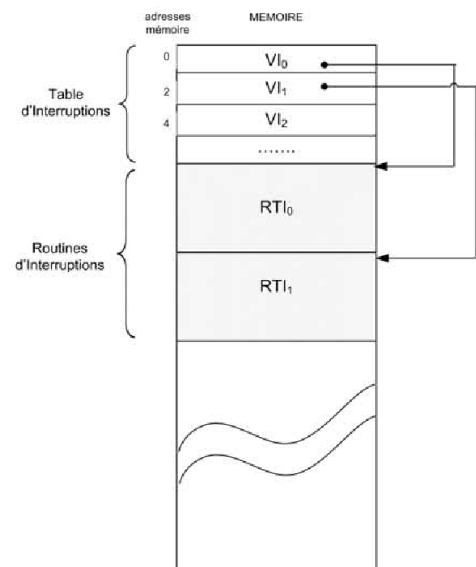


Figure 5 : Table et routines d'interruptions

4.4. Contrôleur d'interruption : Le *contrôleur d'interruptions* est un circuit spécial de la carte mère, extérieur au processeur, dont le rôle est de distribuer et de mettre en attente les demandes d'interruptions provenant de différents périphériques (voir figure 6).

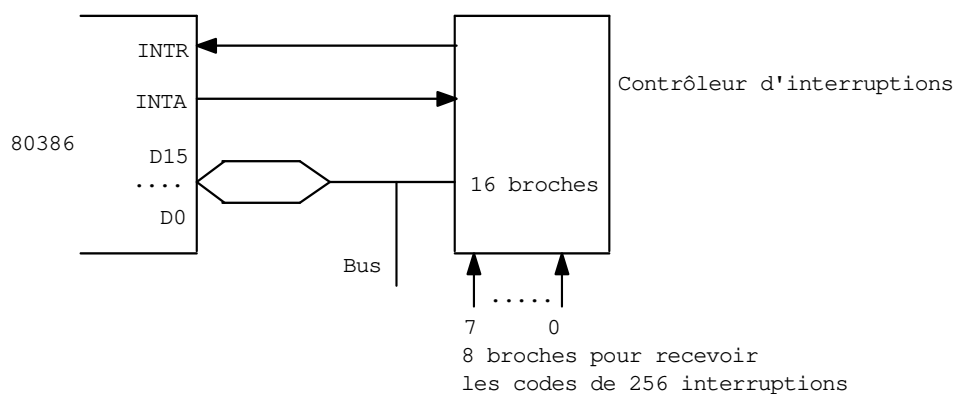


Figure 6 : Contrôleur d'interruptions

- Le contrôleur est relié aux interfaces gérant les périphériques par les **bornes IRQ** (**InterRupt reQuest**). Il gère les demandes d'interruption envoyées par les périphériques, de façon à les envoyer une par une au processeur (via **INTR**).
- Avant d'envoyer la prochaine l'interruption, le contrôleur attend d'avoir reçu le signal **INTA**(**Interrupt acknowledge**), indiquant que le processeur a bien traité l'interruption en cours.

4.5. Registre d'interruption : Chaque catégorie d'interruption est connue par le système, on associe un bit à chacune, cela forme ce que l'on appelle le registre d'interruption. On permet à chacune d'interruption de modifier seulement son bit.

5. Catégories d'interruptions :

Les interruptions peuvent être de diverses origines, mais on peut les classer en deux grandes classes (ou catégories) :

- a) Les interruptions **externes** ou **matérielles**,
- b) Les interruptions **internes** ou **logicielles**.

5.1. Interruptions externes (indépendantes du processus) :

Ce sont les interruptions causées par des organes externes au processeur central, comme les horloges de temps, les périphériques d'E/S, ...etc.

5.2. Interruptions internes :

Ce sont des interruptions causées par des anomalies dans l'exécution du programme en cours et elles sont divisées en deux sous-classes :

5.2.1. Déroutements (**Trap** en anglais ou **Exception** en termes de langage de programmation comme en ADA, JAVA et Python) : Quand un indicateur de changement d'état est modifié par une cause liée à l'exécution d'une instruction en cours, on dit qu'il y a un **déroutement** (*Trap*). Le mécanisme de déroutement a essentiellement pour rôle de traiter une anomalie dans le déroulement d'une instruction :

- o Données incorrectes (débordement arithmétique, division par zéro, etc.)
- o Tentative d'exécution d'une opération interdite par un dispositif de protection (violation de la mémoire, l'exécution d'une instruction privilégiée en mode esclave).

Remarque : Etant donné le caractère synchrone du déroutement, la notion de *masquage* n'a pas de sens à ce niveau, un déroutement peut être supprimé (non traité) mais non retardé.

5.2.2. Appels au superviseur (SuperVisor Call ou SVC) : Ce sont des instructions qui permettent à l'utilisateur d'accéder, depuis son programme, à un service particulier du système d'exploitation. Le mécanisme de cet appel commence par vérifier les droits de l'utilisateur à effectuer cette demande, puis la validité des paramètres transmis. Ce mécanisme permettra à l'utilisateur, par le biais de la procédure système appelée, de fonctionner en mode maître tout en protégeant le système d'actions malveillantes. Donc, Cette façon de procéder permet au système de :

- Se protéger des usagers ;
- Vérifier les droits d'accès au service demandé.

L'appel au superviseur est en fait un appel de procédure système avec un passage de paramètres. Le service demandé par l'utilisateur sera choisi au moyen d'un paramètre supplémentaire désignant la cause de l'appel (voir figure 7).

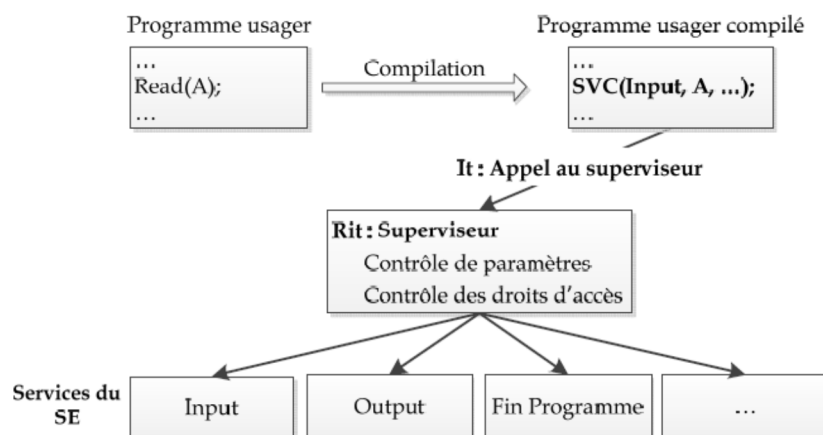


Figure 7 : Exemple d'Appel au superviseur

Un SVC est traité comme **interruption** pour permettre une commutation du contexte, ainsi, il a les avantages suivants :

- L'interruption provoque un branchement vers le traitant d'interruption mais aussi un changement de mode (Esclave→Maître). Il y a donc un passage automatique du programme utilisateur qui s'exécute en mode esclave vers le Traitant_SVC qui s'exécute en mode maître.

- Il existe un et un seul point d'entrée vers le SE pour les processus utilisateur. Il est donc plus facile (du point de vue du concepteur du système) de sécuriser l'appel des routines système.
 - Si on part du principe que le vecteur d'interruptions se trouve dans une zone inaccessible au programme utilisateur, alors ce dernier n'a aucun moyen de passer en mode maître et l'instruction SVC est le seul point de passage.
- Le traitement de l'interruption SVC a la structure suivante :

Structure d'un SVC :
1. sauver le contexte du programme demandeur
2. vérifier la nature de la requête
3. vérifier les arguments (paramètres) de la requête
4. vérifier les droits du demandeur
5. exécuter la demande
6. restaurer le contexte du demandeur
7. retour vers le demandeur

6. Priorité et Système hiérarchisé d'interruptions :

La notion de priorité affectée à un niveau d'interruption découle du fait que les différents équipements nécessitant un traitement d'interruption ne requièrent généralement pas la même urgence de traitement de la part de l'UC : on admettra aisément qu'une interruption émanant d'une unité d'E/S lente telle qu'un clavier ne peut pas avoir la même urgence de prise en compte que celle émanant d'une unité d'E/S rapide telle qu'un disque. Le mécanisme d'interruption est donc construit de telle sorte que chaque niveau d'interruption a une priorité fixée d'avance, dans un intervalle allant de la priorité la plus faible (ou plus basse) à la priorité la plus forte (ou plus élevée). Arbitrairement, les concepteurs d'ordinateurs ont choisi d'allouer la priorité la plus élevée à l'interruption de niveau 0 et la plus faible à l'interruption de niveau N, pour une machine qui dispose de N+1 niveaux d'interruption.

On peut envisager une hiérarchie de priorité des interruptions. Ainsi, lorsque plusieurs interruptions arrivent en même temps, c'est la plus prioritaire qui est prise en charge en premier. Cela implique aussi que la routine elle-même d'une interruption peut être interrompue par une interruption plus prioritaire (voir figure 8).

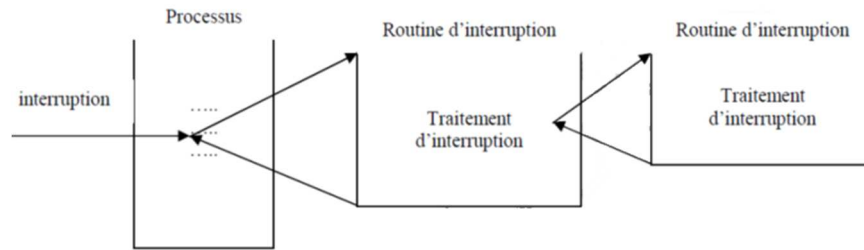


Figure 8 : Interruptions en cascade

Les fils d'interruptions peuvent être *hiérarchisés* c'est-à-dire classés par ordre de priorités respectives. Un traitement d'interruption peut donc être lui-même interrompu par une demande d'interruption intervenant sur un fil de priorité supérieure. Il passe alors à l'état d'attente. La figure 9 représente l'activité des programmes dans le temps pour un système hiérarchisé à 8 niveaux où le niveau 0 est le plus prioritaire, le niveau 7 correspondant au niveau moins prioritaire (programme d'arrière-plan).

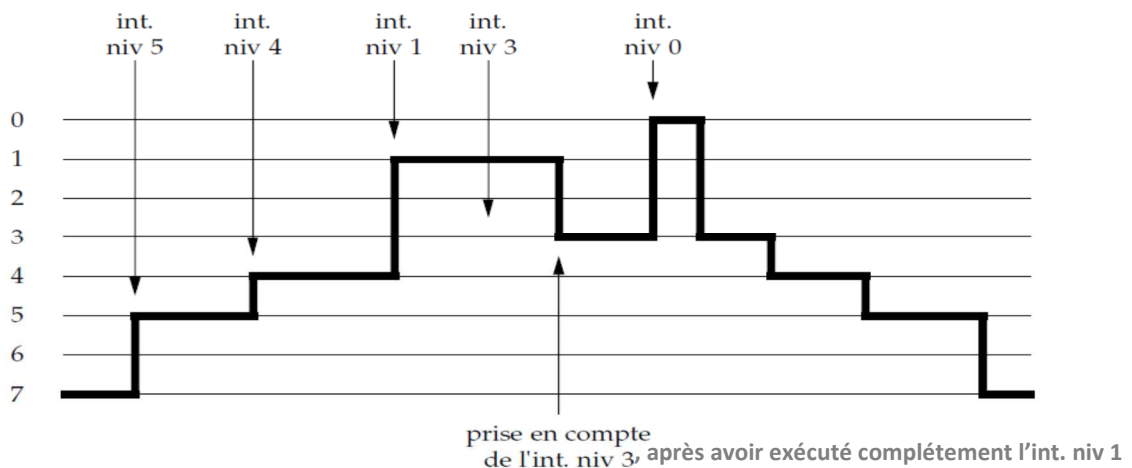


Figure 9 : Effet de la hiérarchisation d'un système d'interruptions

Chaque niveau d'interruption peut être dans l'un des états suivants :

- **état Masqué** : Lorsque le nombre de niveaux d'interruption est limité, un registre spécialisé de la CPU contient ce que l'on appelle le *masque d'interruption*. A chaque niveau est associé un bit indiquant s'il est autorisé ou masqué. Donc dans l'état masqué le niveau est inhibé par programme de sorte que l'interruption a pu être mémorisée mais ne peut être prise en compte par la CPU.

Une interruption masquée ne peut interrompre le processeur, mais toute demande d'interruption faite durant le masquage est retardée (mémorisée) pour être traitée à la levée

du masquage. Lors du traitement d'une interruption, les interruptions moins prioritaires sont automatiquement et temporairement désactivées. Ainsi, les interruptions de même niveau de priorité ou d'un niveau plus bas peuvent être masquées, alors qu'une interruption de priorité supérieure est en cours d'exécution.

- **état Armé / Désarmé** : Une interruption d'un niveau **désarmée** ne peut interrompre le processeur. Ceci se passe comme si la cause de l'interruption était supprimée. Toute demande d'interruption faite durant son désarmement est perdue. On utilise ce procédé quand on désire qu'un élément ne doive plus interrompre. On peut *armer* ou *désarmer* un niveau d'interruption par programme en utilisant des *instructions privilégiées*. Cette possibilité est donc réservée au SE.
- **état d'attente** : l'interruption peut être prise en compte immédiatement si deux conditions sont remplies :
 - aucun niveau de priorité supérieure n'est en état d'attente ;
 - la CPU se trouve dans un point interruptible.

7. Conclusion :

La gestion des interruptions est un élément clé des systèmes d'exploitation modernes, permettant une **réactivité optimale du processeur** face aux événements internes et externes. En comprenant leur fonctionnement, nous avons vu comment elles permettent d'améliorer la performance du système tout en assurant une gestion efficace des ressources.

Le **mot d'état du processeur (PSW)** joue un rôle crucial dans la sauvegarde et la restauration du contexte des processus interrompus, garantissant ainsi la continuité de l'exécution sans perte d'information. De même, l'**instruction LPSW** permet de rétablir un contexte d'exécution, assurant une reprise correcte des tâches après le traitement d'une interruption.

Enfin, la gestion des interruptions repose sur un **équilibre entre réactivité et optimisation** : un bon système d'exploitation doit **prioriser les interruptions critiques**, éviter les interruptions inutiles et minimiser l'impact des commutations de contexte sur les performances globales du processeur. Une gestion efficace des interruptions est donc essentielle pour garantir **la stabilité, la rapidité et la fiabilité** d'un système informatique.

Ce chapitre a fourni un aperçu détaillé des différents aspects de la gestion des interruptions et a fourni une base solide pour poursuivre l'étude des systèmes d'exploitation.

Schéma de synthèse résumant le chapitre

